```
assert = require 'assert'
{addWord} = require './word_utils'

do addWordShouldAddOneWord = ->
  input = "ultra mega"
  expectedOutput = "ultra mega ok"
  actualOutput = addWord input, "ok"
  assert.equal expectedOutput, actualOutput
```

Require the implementation from inside the test.

Make it pass

Implement the `addWord` function to make the test pass. The word_utils.coffee file initially `exports` an empty `addWord` function:

```
addWord = (existing, addition) -> # not implemented
exports.addWord = addWord
```

In practice you'd now continue to implement a working solution. This time, though, pause for a moment and think, what would happen if you ran the test now? It should fail because the `addWord` function returns `undefined`. To make sure the program is currently working how you think it's working, you run the test:

```
> coffee word_utils.spec.coffee
AssertionError: "ultra mega ok" == "undefined"
```

It fails exactly as you expected it to. *That's good*. You now implement a working `addWord` function:

```
addWord = (text, word) ->
  "#{text} #{word}"
```

You run the test and it passes:

```
> coffee word_utils.spec.coffee
# No output
```

The common.js assertions return `undefined` when there's no failure, so there's no console output when this test passes.